

Problem Statement 1:

1. API Data Retrieval and Storage: You are tasked with fetching data from an external REST API, storing it in a local SQLite database, and displaying the retrieved data. The API provides a list of books in JSON format with attributes like title, author, and publication year.

Used the Open Library REST API to retrieve book data in JSON format.

- Selected the required fields: book key, title, author, and publication year.
- Normalized the API response and handled missing values.
- Created a SQLite database named books.db
- Created a books table with source_id as the primary key.
- Added NOT NULL, default, and nullable constraints as required.
- Added indexes for author and publication year.
- Used parameterized SQL and UPSERT to insert/update records without duplicates.
- Retrieved the stored records from SQLite and displayed them.

2. Data Processing and Visualization: Given a dataset containing information about students' test scores, fetch the data from an API, calculate the average score, and create a bar chart to visualize the data.

- Fetched student exam-score data from a public API.
- Extracted student names and exam scores from the API response.
- Converted score values to numeric format and handled missing/invalid values.
- Calculated the average exam score using the cleaned dataset.
- Created a horizontal bar chart to clearly visualize all students' exam scores.
- Used a fixed 0–100 range for the visualization.

3. CSV Data Import to a Database: Write a Python script that reads data from a CSV file containing user information (e.g., name, email) and inserts it into a SQLite database.

- Created and read a CSV file containing user information.
- Used csv.DictReader to read the records correctly.
- Validated required fields such as name and email.
- Performed basic email validation.
- Handled missing/unparseable age values by storing them as NULL.
- Created a SQLite users table with primary key, NOT NULL, and UNIQUE constraints.
- Used email as the natural key with case-insensitive uniqueness.
- Inserted the validated records using UPSERT.
- Retrieved and displayed the records stored in the SQLite database.

LINK TO COLAB:

<https://colab.research.google.com/drive/1dRAC3Ec7L4z0MIAdhdiN9nMSbMtIl-Ix?usp=sharing>

4. LINK TO COMPLEX PYTHON CODE:

<https://leetcode.com/problems/search-a-2d-matrix/description/>
<https://leetcode.com/problems/trapping-rain-water/description/>

5. LINK TO COMPLEX DATABASE CODE:

<https://leetcode.com/problems/managers-with-at-least-5-direct-reports/>

Problem Statement 2:

1. Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

LLM,AI,ML- B

Deep Learning-C

2. What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level

In short: there are three ways to get knowledge into a model — fine-tune it, stuff the whole document into the prompt, or retrieve the relevant parts. I chose retrieval (RAG). That decision gives you two separate pipelines meeting at a vector store, six components, and a grounding check before generation.

How the model gets its knowledge

Fine-tuning writes knowledge into the weights. It's expensive, slow to update, and pointless when every user uploads a different PDF — there's no stable corpus to train on.

Long-context stuffing sends the whole document with every request. Simple, but cost and latency grow with document size, and it breaks once the document exceeds the context window.

RAG keeps the document in a store, pulls only the passages that match the question, and puts those in the prompt. Cost stays flat regardless of document size, new documents are usable the moment they're uploaded, and every answer traces back to a passage.

I went with RAG. Everything below follows from that.

Two pipelines, not one

A RAG chatbot isn't a single request path. It's two independent pipelines that meet at a shared vector store, and they behave very differently.

The **write path** runs once per uploaded document. It's slow — a few seconds — and that's acceptable, because people expect an upload to take time.

The **read path** runs on every message. It has to be fast, and it only reads.

Once you see the split, the structure follows: separate endpoints, separate error handling, separate things to optimise. Treating it as one linear flow is the usual mistake.

The six components

1. Interface. A single Next.js page with an upload area and a chat panel. It holds conversation history in component state and contains no business logic — it calls the API and renders what comes back. Each answer has an expandable Sources section showing the retrieved passages and their scores, so retrieval quality is visible instead of hidden.

2. Orchestration. Two Next.js API routes: `/api/upload` owns the write path, `/api/chat` owns the read path. Both run on Node inside the same app, so there's no separate backend to deploy. All credentials live here — the OpenAI key and the Supabase service role key never reach the browser.

3. Ingestion. Four stages that turn a document into searchable vectors:

- *Extract* — pdf-parse pulls plain text out of the PDF.
- *Chunk* — LangChain's RecursiveCharacterTextSplitter cuts it into 500-character chunks with 50 characters of overlap. Chunking matters because an embedding is a fixed-size summary of whatever you give it, so embedding a whole document produces a vector too vague to match any specific question. The overlap stops a sentence that straddles a boundary from being split across two vectors, where it would match neither well.
- *Embed* — each chunk goes to OpenAI's embedding model and comes back as a 1,536-dimensional vector. Chunks are batched into single API calls rather than one request each.
- *Persist* — chunks and vectors are batch-inserted with a `document_id` foreign key.

4. Storage. Supabase Postgres with pgvector. A documents table for file metadata, and a chunks table holding the chunk text, its `document_id`, and a `vector(1536)` column, with an HNSW index over the embeddings. Metadata and vectors sit in one database — reasoning in Q2.

5. Retrieval. The question is embedded with the same model used for the chunks, then a cosine similarity search returns the five closest chunks. That search runs inside a Postgres function, `match_chunks`, rather than in application code, so the index does the work and the database returns five rows instead of the whole table.

6. Generation. The retrieved passages go into a system prompt telling the model to answer strictly from that context, and the sources come back attached to the answer. The point is

that the model is reading, not recalling — the answer comes from the prompt, not the weights.

Control flow and guardrails

The query pipeline is a LangGraph state graph rather than a chain of function calls. Four nodes — `embedQuestion`, `retrieve`, `generate`, `noContext` — with a conditional edge after retrieval. Each stage is inspectable on its own, so when a bad answer shows up I can tell which stage caused it instead of picking apart one long function.

That conditional edge is the guardrail. If the best match is too far away — cosine distance above 0.3, roughly 0.7 similarity — the graph routes to `noContext` and returns "this document doesn't appear to cover that" without calling the LLM at all. That's the line between a grounded system and one that answers confidently from general knowledge when retrieval has failed. It also saves an API call.

One invariant that has to hold

Chunks and questions must be embedded by the same model. Different models place vectors in different spaces, so mixing them makes the similarity scores meaningless. It's a nasty failure mode because nothing errors — the system keeps running and quietly retrieves the wrong passages, which are then handed to the model as authoritative context. Changing the embedding model means re-embedding the whole corpus.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

In short: a vector database stores embeddings and answers nearest-neighbour queries efficiently. For my problem — per-document Q&A over uploaded PDFs, a corpus in the hundreds of thousands of vectors, with strict per-document scoping — I chose pgvector on Postgres over Pinecone, Qdrant, Weaviate, Milvus and Chroma.

What a vector database is

An embedding model turns text into a high-dimensional vector — 1,536 dimensions in my case — positioned so that similar meanings land near each other geometrically. Meaning becomes distance, so "find text that means something similar" turns into "find the nearest points."

A vector database stores those vectors and answers nearest-neighbour queries over them.

That needs different machinery from a normal database index. B-trees work for WHERE id = 5 or WHERE date > '2026-01-01' because there's a total order over the values. There's no meaningful total order in 1,536-dimensional space, so B-trees don't help. Comparing the query vector against every stored vector is exact but scales linearly, which stops being practical as the corpus grows.

The answer is approximate nearest neighbour (ANN) indexes, which return *probably* the closest matches in sub-linear time. The central trade-off in the field is recall against latency against memory: give up a little exactness for a lot of speed.

Distance metrics

- **Cosine** — the angle between vectors, ignoring magnitude. The standard for text.
- **Euclidean (L2)** — straight-line distance, sensitive to magnitude.
- **Inner product** — dot product, common in recommendation models.

OpenAI embeddings are normalised to unit length, so all three produce identical rankings on my data. I used cosine because it's the convention for text and reads clearly in the query.

Index types

- **Flat / brute force** — compare against everything. Exact, no build cost, linear query time. Correct for small collections.
- **IVFFlat** — clusters the vectors and searches only the nearest clusters. Fast to build and light on memory, but it needs representative training data and loses recall near cluster boundaries.
- **HNSW** — a multi-layer proximity graph traversed greedily from a sparse top layer down. Best recall-to-latency ratio in practice, at the cost of slower builds and more memory.
- **Product quantisation** — compresses vectors to shrink memory, giving up precision. Used at very large scale.

The problem I'm solving

A document Q&A service where people upload PDFs and query them conversationally. The requirements:

Scale. At 500-character chunks with 50 overlap, a 50-page PDF works out to roughly 250–350 chunks (assuming around 2,500 characters a page). A few thousand documents puts the corpus at 10^5 – 10^6 vectors. At 1,536 float4 dimensions each vector is about 6 KB, so 100,000 chunks is roughly 600 MB of vector data.

Strict scoping. Every search must stay inside one document. Returning a passage from someone else's PDF isn't a ranking flaw, it's a correctness and privacy failure. This is the hardest requirement.

Relational metadata. Filenames, upload timestamps, and eventually user ownership, all joined against chunks.

Latency budget. Generation dominates the response at one to three seconds, so retrieval has tens of milliseconds to play with.

Team. One developer, no dedicated infrastructure support.

The choice: pgvector on Postgres, with HNSW

Worth saying plainly: pgvector is a Postgres extension, not a standalone vector database. Choosing it over a purpose-built service is the actual decision here, and it rests on four things.

1. The scale doesn't justify a dedicated service. Below roughly a million vectors, pgvector with HNSW answers in single-digit to low-tens of milliseconds — comfortably inside a budget dominated by the LLM. Pinecone, Qdrant, Weaviate and Milvus earn their operational cost at tens of millions of vectors or high sustained query throughput. Adopting one here means paying for headroom I don't use.

2. My hardest requirement is the thing Postgres does best. Document scoping is `WHERE document_id = $1` — an ordinary indexed relational filter the query planner already understands, applied with exact correctness. In a dedicated vector database, metadata filtering is a first-class feature but a more delicate one: pre-filtering can degrade the ANN graph traversal and hurt recall, while post-filtering can return fewer results than requested.

3. One datastore removes a consistency problem. Vectors and metadata share a transaction boundary, so `ON DELETE CASCADE` removes a document's chunks atomically. A separate vector database means dual writes to two systems, and any failed write leaves orphaned vectors — which later surface as retrieved context from a document the user believes they deleted. Not having a distributed consistency problem at all is worth more here than any performance gain.

4. Operational simplicity. No extra service, no extra credentials, no separate backup strategy. The vector store is backed up when the database is.

Within pgvector I picked HNSW over IVFFlat because the corpus is small enough that memory isn't a constraint, read performance matters more than index build time, and IVFFlat's need for representative training data is awkward when the collection grows one upload at a time.

When I'd switch, and to what

Past roughly a million vectors, or if I needed heavy hybrid search, multi-tenant sharding, or high sustained QPS, I'd move to **Qdrant** — it's fast, self-hosts cleanly, and its payload filtering holds up well under ANN search, which matters given how central document scoping is for me. **Pinecone** if I wanted it fully managed and was willing to pay for that. **Milvus** at genuinely large scale, with a team to operate it. **Chroma** is fine for prototyping but I wouldn't put it under this.